

INFORME DE PENTESTING Y HARDENING



Análisis de Seguridad, Pentesting y Hardening

Servidor Debian con WordPress

Autor

Gabriel Monsalve Rodríguez

1. RESUMEN EJECUTIVO

1.1 Objetivo del proyecto

El presente proyecto tuvo como objetivo principal realizar un análisis de seguridad ofensiva y defensiva sobre un servidor Debian con servicios web desplegados mediante Apache2, MariaDB y WordPress.

La práctica se centró en:

- identificar vulnerabilidades de seguridad,
- analizar evidencias forenses,
- explotar configuraciones inseguras,
- aplicar medidas de mitigación,
- endurecer el sistema (hardening),
- validar la persistencia de las correcciones,
- y diseñar controles de seguridad alineados con buenas prácticas de ciberseguridad e ISO 27001.

Asimismo, se buscó desarrollar procedimientos de respuesta ante incidentes y documentación técnica profesional basada en evidencias reales obtenidas durante el laboratorio.

1.2 Resumen de vulnerabilidades detectadas

Durante el análisis del entorno se identificaron múltiples vulnerabilidades y configuraciones inseguras que comprometían la confidencialidad, integridad y disponibilidad del sistema.

Las principales vulnerabilidades detectadas fueron:

Vulnerabilidad

Acceso SSH inseguro
Login root habilitado
Autenticación por contraseña SSH
FTP inseguro
Posible acceso anonymous FTP
Password débil
WordPress/MySQL
Permisos inseguros en wp-config.php
XMLRPC habilitado

Riesgo asociado

Acceso remoto no autorizado
Escalada total de privilegios
Ataques de fuerza bruta
Exposición de credenciales
Acceso no autorizado
Compromiso de base de datos
Exposición de credenciales críticas
Fuerza bruta y abuso de

Directory Listing en Apache	pingbacks
Servicios innecesarios activos	Enumeración de archivos internos
	Incremento superficie de ataque

Además, se detectó exposición innecesaria de servicios como:

- Avahi,
 - CUPS,
 - Exim4,
- los cuales no eran necesarios para la funcionalidad del servidor web.

1.3 Medidas de mitigación aplicadas

Tras identificar las vulnerabilidades, se aplicaron múltiples medidas de hardening y mitigación orientadas a reducir la superficie de ataque y reforzar la seguridad del sistema.

Las principales medidas implementadas fueron:

Medida aplicada	Objetivo
Deshabilitación de root login SSH	Evitar acceso privilegiado remoto
Deshabilitación PasswordAuthentication	Mitigar fuerza bruta SSH
Sustitución de contraseña débil	Proteger WordPress y MariaDB
Restricción permisos wp-config.php (0600)	Proteger credenciales críticas
Deshabilitación FTP/vsftpd	Eliminar exposición FTP
Deshabilitación XMLRPC	Mitigar ataques WordPress
Apache Options -Indexes	Evitar enumeración directorios
Eliminación servicios innecesarios	Reducir superficie ataque
Validación post-reinicio	Garantizar persistencia mitigaciones

También se realizaron múltiples verificaciones técnicas posteriores para confirmar la correcta aplicación de las medidas y asegurar que los cambios persistieran tras reiniciar el sistema.

1.4 Resultados obtenidos

Como resultado del proceso de hardening y mitigación, el servidor redujo significativamente su superficie de ataque y mejoró su postura de seguridad.

Las validaciones finales confirmaron que:

- los servicios innecesarios quedaron completamente deshabilitados,
- los puertos inseguros fueron cerrados,
- SSH quedó endurecido,
- WordPress mantuvo su disponibilidad operativa,
- las credenciales críticas quedaron protegidas,
- y las mitigaciones persistieron correctamente tras reiniciar la máquina virtual.

El estado final del sistema mostró únicamente los servicios estrictamente necesarios:

- Apache (HTTP),
- SSH endurecido,
- y MariaDB restringido a localhost.

Asimismo, se generó documentación técnica completa incluyendo:

- evidencias forenses,
- validaciones,
- capturas de pantalla,
- scripts,
- y procedimientos de mitigación, permitiendo mantener trazabilidad y soporte documental del proyecto realizado.

2. INTRODUCCIÓN

2.1 Contexto del proyecto

En la actualidad, los entornos web y servidores Linux representan uno de los principales objetivos de ataques informáticos debido a la gran cantidad de servicios

expuestos y configuraciones inseguras que pueden comprometer la seguridad de una organización.

La presencia de servicios vulnerables, credenciales débiles, configuraciones por defecto y mecanismos de autenticación inseguros puede facilitar ataques de fuerza bruta, accesos no autorizados, escaladas de privilegios y exposición de información sensible.

El presente proyecto se desarrolló sobre un entorno basado en Debian Linux con servicios Apache2, MariaDB y WordPress, simulando una infraestructura web empresarial vulnerable susceptible de ser comprometida mediante distintas técnicas de enumeración y explotación.

Durante el desarrollo de la práctica se realizó:

- análisis forense,
- identificación de vulnerabilidades,
- explotación controlada,
- aplicación de medidas de hardening,
- mitigación de riesgos,
- y validación posterior de la seguridad implementada.

Asimismo, se documentó todo el proceso técnico mediante evidencias, capturas y validaciones reales, siguiendo buenas prácticas de ciberseguridad defensiva y respuesta ante incidentes.

El proyecto también incorpora conceptos relacionados con:

- hardening de sistemas Linux,
- protección de aplicaciones web,
- gestión de accesos,
- reducción de superficie de ataque,
- análisis de riesgos,
- y alineación con controles inspirados en ISO 27001 y NIST SP 800-61.

2.2 Objetivos técnicos

El objetivo principal del proyecto fue identificar y mitigar vulnerabilidades presentes en un servidor Debian vulnerable, aplicando procedimientos de seguridad ofensiva y defensiva orientados a reforzar la protección del entorno.

Los objetivos técnicos específicos fueron los siguientes:

- Identificar servicios expuestos y configuraciones inseguras del sistema.

- Realizar enumeración de puertos y superficie de ataque mediante herramientas de análisis.
- Detectar vulnerabilidades relacionadas con:
 - SSH,
 - FTP,
 - Apache,
 - WordPress,
 - MariaDB,
 - permisos inseguros,
 - y servicios innecesarios.
- Analizar evidencias forenses y actividad del sistema.
- Validar riesgos asociados a credenciales débiles y configuraciones inseguras.
- Aplicar medidas de hardening sobre el servidor Linux.
- Implementar controles de seguridad para proteger servicios críticos.
- Reducir la superficie de ataque mediante la deshabilitación de servicios innecesarios.
- Proteger archivos sensibles como wp-config.php mediante controles de permisos.
- Deshabilitar componentes inseguros de WordPress como XMLRPC.
- Endurecer el acceso remoto SSH eliminando mecanismos inseguros de autenticación.
- Validar técnicamente todas las mitigaciones implementadas.
- Garantizar la persistencia de las configuraciones tras reiniciar el sistema.
- Documentar todo el proceso técnico mediante evidencias reales y capturas de pantalla.
- Desarrollar un enfoque básico de SGSI alineado con controles de ISO 27001.
- Diseñar procedimientos básicos de respuesta a incidentes basados en NIST SP 800-61.

Finalmente, el proyecto tuvo como propósito demostrar la importancia de la correcta configuración de sistemas Linux y aplicaciones web, así como la necesidad de aplicar controles de seguridad continuos para minimizar riesgos y proteger la infraestructura frente a amenazas reales.

3. DESCRIPCIÓN DE LA INFRAESTRUCTURA

3.1 Sistema operativo Debian

La infraestructura del laboratorio se desplegó sobre una máquina virtual con sistema operativo Debian Linux, utilizado como servidor principal para alojar los servicios web y de base de datos del entorno vulnerable.

Debian fue seleccionado debido a:

- su estabilidad,
- amplia utilización en entornos empresariales,
- soporte para servicios web,
- y facilidad para realizar tareas de administración y hardening.

La máquina virtual fue desplegada en un entorno VirtualBox utilizando una red local virtualizada para permitir la comunicación entre:

- la máquina atacante,
- la máquina host,
- y el servidor objetivo.

Durante el proyecto se realizaron tareas de:

- administración Linux,
- análisis forense,
- gestión de servicios,
- hardening,
- configuración de permisos,
- y validación de seguridad sobre el sistema Debian.

Asimismo, el servidor fue configurado con múltiples servicios expuestos intencionadamente para simular un entorno vulnerable susceptible de análisis y mitigación.

3.2 Apache2

Apache2 fue utilizado como servidor web principal encargado de alojar el entorno WordPress desplegado sobre la máquina Debian.

El servicio HTTP se encontraba inicialmente expuesto mediante el puerto 80/tcp y permitió:

- servir contenido web,
- alojar WordPress,
- y validar configuraciones relacionadas con seguridad web y hardening.

Durante las fases del proyecto se identificaron configuraciones inseguras relacionadas con Apache, incluyendo:

- listado automático de directorios (Directory Listing),
- exposición de archivos internos,

- y presencia de archivos por defecto del servidor.

Como parte del proceso de mitigación se aplicaron medidas de hardening mediante:

- deshabilitación de “Options Indexes”,
- validación de directorios accesibles,
- y comprobaciones posteriores de disponibilidad del servicio web.

Las validaciones finales confirmaron que Apache continuó operativo tras aplicar las medidas de seguridad implementadas.

3.3 MariaDB

MariaDB fue utilizada como sistema gestor de bases de datos encargado de almacenar la información asociada al entorno WordPress.

La base de datos se encontraba accesible localmente mediante el puerto:

- 3306/tcp.

Durante el análisis de seguridad se identificó el uso de credenciales débiles dentro del archivo:

- wp-config.php,
lo cual representaba un riesgo crítico de acceso no autorizado a la base de datos.

Como medida de mitigación:

- se sustituyeron las credenciales inseguras,
- se reforzaron los permisos de acceso al archivo de configuración,
- y se validó que MariaDB permaneciera accesible únicamente desde localhost.

Asimismo, se aplicaron controles orientados a proteger:

- la confidencialidad de la información,
- las credenciales del sistema,
- y la integridad de la configuración del entorno WordPress.

3.4 WordPress

WordPress fue desplegado como aplicación web principal alojada sobre Apache2 y respaldada mediante MariaDB.

El entorno WordPress fue utilizado para:

- simular un servicio web empresarial real,
- identificar vulnerabilidades comunes en aplicaciones web,
- y aplicar controles de hardening sobre componentes críticos.

Durante el proyecto se detectaron múltiples riesgos relacionados con WordPress, incluyendo:

- uso de credenciales débiles,
- permisos inseguros sobre wp-config.php,
- exposición del endpoint xmlrpc.php,
- y configuraciones inseguras del servidor web.

Como parte de las medidas de seguridad implementadas:

- se protegió el archivo wp-config.php mediante permisos restrictivos,
- se deshabilitó XMLRPC,
- y se validó el correcto funcionamiento del sitio web tras aplicar las mitigaciones.

Las verificaciones posteriores confirmaron que WordPress permaneció operativo y accesible tras completar el proceso de hardening.

3.5 Servicios adicionales instalados

Además de Apache2, MariaDB y WordPress, el sistema contaba inicialmente con diversos servicios adicionales instalados y activos, algunos de los cuales incrementaban innecesariamente la superficie de ataque del servidor.

Entre los servicios identificados se encontraban:

- SSH,
- vsftpd (FTP),
- Avahi,
- CUPS,
- Exim4,
- y otros servicios auxiliares del sistema.

Durante el análisis de seguridad se comprobó que algunos de estos servicios:

- no eran necesarios para el funcionamiento del entorno web,
- exponían puertos innecesarios,
- o podían facilitar vectores de ataque adicionales.

Como medida de hardening:

- se deshabilitaron servicios innecesarios,
- se cerraron puertos expuestos,
- y se validó la persistencia de las mitigaciones tras reiniciar el sistema.

El objetivo principal de estas acciones fue reducir la superficie de ataque y limitar los posibles vectores de compromiso del servidor.

4. FASE 1 — ANÁLISIS FORENSE Y RECOLECCIÓN DE EVIDENCIAS

4.1 Identificación del incidente

Durante la fase inicial del proyecto se realizó un análisis de seguridad sobre el servidor Debian con el objetivo de identificar posibles indicios de compromiso, configuraciones inseguras y servicios vulnerables expuestos en la infraestructura.

El entorno presentaba múltiples configuraciones deficientes que podían facilitar:

- accesos no autorizados,
- escaladas de privilegios,
- exposición de información sensible,
- y explotación remota de servicios.

Entre los principales indicadores identificados se encontraban:

- servicios innecesarios activos,
- autenticación SSH insegura,
- FTP expuesto,
- credenciales débiles en WordPress/MariaDB,

- y componentes inseguros habilitados en WordPress.

Asimismo, se detectó exposición de múltiples puertos y servicios que incrementaban significativamente la superficie de ataque del sistema.

El análisis forense se enfocó en:

- identificar evidencias técnicas,
- validar configuraciones vulnerables,
- y determinar posibles riesgos asociados al entorno comprometido.

4.2 Análisis de logs del sistema

Se realizó una revisión básica de logs y eventos del sistema Linux para identificar:

- actividad sospechosa,
- servicios activos,
- accesos remotos,
- y configuraciones potencialmente inseguras.

Durante el análisis se revisaron:

- procesos activos,
- puertos en escucha,
- configuración de servicios,
- permisos críticos,
- y estado de los demonios del sistema.

También se verificó:

- la configuración SSH,
- la exposición de servicios mediante sockets activos,
- y la presencia de servicios innecesarios ejecutándose en segundo plano.

El análisis permitió identificar:

- servicios vulnerables activos,
- exposición innecesaria de puertos,
- y configuraciones por defecto inseguras que requerían mitigación inmediata.

4.3 Evidencias obtenidas

Durante el proceso de análisis forense se recopilaron múltiples evidencias técnicas relacionadas con el estado del sistema y las vulnerabilidades identificadas.

Las principales evidencias obtenidas incluyeron:

- listado de puertos abiertos,
- servicios activos,
- configuraciones inseguras,
- permisos de archivos críticos,
- configuraciones SSH,
- y validaciones posteriores de mitigación.

Entre las evidencias más relevantes se encontraron:

- puerto FTP expuesto,
- autenticación SSH insegura,
- password débil almacenada en wp-config.php,
- exposición de xmlrpc.php,
- y listado automático de directorios Apache.

Asimismo, se generaron:

- capturas de pantalla,
- documentos de validación,
- evidencias forenses,
- y registros técnicos de las mitigaciones aplicadas.

Toda la información recopilada fue documentada para mantener trazabilidad y soporte técnico del proceso de hardening realizado.

4.4 Enumeración de usuarios y privilegios

Durante la fase de reconocimiento y análisis se revisaron configuraciones relacionadas con:

- autenticación,
- privilegios,
- y accesos remotos al sistema.

Se identificó que el servidor SSH permitía inicialmente:

- autenticación mediante contraseña,
- y acceso remoto utilizando cuentas privilegiadas.

Estas configuraciones representaban un riesgo crítico debido a la posibilidad de:

- ataques de fuerza bruta,
- reutilización de credenciales,
- y compromiso total del sistema mediante acceso root remoto.

También se verificaron permisos asociados a archivos críticos como:

- wp-config.php,
el cual contenía credenciales sensibles de WordPress y MariaDB.

Posteriormente se aplicaron controles de seguridad orientados a:

- restringir privilegios,
- proteger archivos críticos,
- y aplicar el principio de mínimo privilegio.

4.5 Identificación de procesos sospechosos

Se realizó una revisión de procesos y servicios activos con el objetivo de detectar:

- servicios innecesarios,
- exposición de aplicaciones vulnerables,
- y posibles vectores de ataque adicionales.

Durante el análisis se identificaron servicios ejecutándose que no eran necesarios para el funcionamiento del entorno web, incluyendo:

- Avahi,
- CUPS,
- Exim4,
- y vsftpd.

La presencia de estos servicios incrementaba la superficie de ataque del servidor y podía facilitar:

- enumeración de red,
- exposición de servicios,

- o explotación de vulnerabilidades conocidas.

Tras el análisis se procedió a:

- deshabilitar servicios innecesarios,
- cerrar puertos asociados,
- y validar la correcta persistencia de las mitigaciones aplicadas.

4.6 Revisión de servicios expuestos

Se realizó una enumeración de puertos y servicios expuestos utilizando herramientas de análisis de red y comandos de inspección locales.

El análisis permitió identificar servicios accesibles mediante:

- HTTP,
- SSH,
- FTP,
- MariaDB,
- y diversos servicios auxiliares del sistema.

Inicialmente el servidor presentaba:

- FTP abierto en el puerto 21/tcp,
- Avahi utilizando el puerto 5353/udp,
- CUPS en el puerto 631/tcp,
- y otros servicios adicionales innecesarios.

La revisión confirmó que varios de estos servicios:

- no eran imprescindibles para la funcionalidad del servidor,
- y aumentaban innecesariamente la superficie de ataque.

Como parte del hardening:

- se deshabilitaron servicios innecesarios,
- se cerraron puertos vulnerables,
- y se validó posteriormente el estado final del sistema.

4.7 Identificación de configuraciones inseguras

Durante el análisis de seguridad se detectaron múltiples configuraciones inseguras relacionadas con:

- autenticación,
- permisos,
- servicios expuestos,
- y componentes vulnerables del entorno WordPress.

Entre las configuraciones inseguras identificadas destacaron:

- acceso SSH mediante contraseña,
- login root habilitado,
- password débil en WordPress/MariaDB,
- permisos inseguros sobre wp-config.php,
- FTP activo,
- XMLRPC habilitado,
- y listado automático de directorios Apache.

Estas configuraciones podían facilitar:

- ataques de fuerza bruta,
- exposición de información sensible,
- acceso remoto no autorizado,
- y compromiso parcial o total del servidor.

Posteriormente se aplicaron medidas correctivas orientadas a eliminar dichas debilidades y reforzar la postura de seguridad del entorno.

4.8 Escaneo de malware y rootkits

Como parte de la fase de análisis forense se realizaron comprobaciones básicas orientadas a detectar posibles indicios de:

- malware,
- procesos maliciosos,
- servicios anómalos,
- o comportamientos sospechosos en el sistema.

Durante el análisis no se identificaron evidencias claras de:

- rootkits activos,
- malware persistente,
- ni procesos desconocidos ejecutándose en segundo plano.

No obstante, sí se detectaron múltiples configuraciones vulnerables y servicios inseguros que podrían facilitar un compromiso futuro del sistema si no eran mitigados correctamente.

El análisis permitió concluir que el principal riesgo del entorno no provenía de malware activo, sino de:

- configuraciones inseguras,
- servicios vulnerables,
- y exposición innecesaria de superficie de ataque.

4.9 Conclusiones del análisis forense

El análisis forense realizado permitió identificar múltiples debilidades de seguridad presentes en el servidor Debian y en los servicios desplegados sobre la infraestructura.

Las principales conclusiones obtenidas fueron:

- existencia de configuraciones inseguras críticas,
- exposición innecesaria de servicios,
- credenciales débiles,
- y componentes vulnerables habilitados en el entorno WordPress.

Asimismo, se comprobó que la infraestructura presentaba una superficie de ataque elevada debido a:

- puertos abiertos innecesarios,
- autenticación remota insegura,
- y servicios no requeridos activos.

Aunque no se detectaron evidencias directas de malware o rootkits activos, el entorno presentaba múltiples vectores potenciales de explotación que podían comprometer la confidencialidad, integridad y disponibilidad del sistema.

Como resultado del análisis se procedió posteriormente a:

- aplicar medidas de hardening,
- corregir configuraciones inseguras,
- reducir la superficie de ataque,

- y validar técnicamente todas las mitigaciones implementadas.

5. FASE 2 — DETECCIÓN Y EXPLOTACIÓN DE VULNERABILIDADES

5.1 Escaneo de puertos y servicios

Durante esta fase se realizó un proceso de reconocimiento activo sobre la infraestructura con el objetivo de identificar:

- puertos abiertos,
- servicios expuestos,
- versiones instaladas,
- y posibles vectores de ataque presentes en el entorno.

Para ello se utilizaron herramientas de enumeración y escaneo como:

- Nmap,
- comandos de inspección locales,
- y herramientas de análisis de servicios Linux.

Los escaneos permitieron identificar servicios accesibles mediante:

- HTTP,
- SSH,
- FTP,
- MariaDB,
- y diversos servicios auxiliares expuestos por el sistema.

Asimismo, se detectaron múltiples puertos abiertos y servicios innecesarios activos que incrementaban considerablemente la superficie de ataque del servidor.

5.2 Enumeración de superficie de ataque

La fase de enumeración permitió identificar todos los componentes accesibles desde red y analizar posibles configuraciones inseguras asociadas a los servicios desplegados.

Entre los principales elementos identificados se encontraban:

- servidor Apache2,

- WordPress,
- MariaDB,
- SSH,
- FTP,
- y diversos servicios auxiliares activos en segundo plano.

El análisis de superficie de ataque confirmó:

- exposición innecesaria de servicios,
- configuraciones por defecto inseguras,
- y presencia de componentes susceptibles de explotación.

Además, se identificaron:

- archivos críticos accesibles,
- componentes inseguros habilitados,
- y permisos deficientes sobre información sensible.

La enumeración permitió establecer las prioridades de mitigación y orientar posteriormente el proceso de hardening del sistema.

5.3 Vulnerabilidades detectadas

Durante el análisis se detectaron múltiples vulnerabilidades y configuraciones inseguras que comprometían la seguridad del entorno.

Las principales vulnerabilidades identificadas fueron:

- autenticación SSH insegura,
- FTP expuesto,
- password débil en WordPress/MariaDB,
- permisos inseguros sobre archivos críticos,
- XMLRPC habilitado,
- listado automático de directorios Apache,
- y servicios innecesarios activos.

Estas vulnerabilidades permitían potencialmente:

- acceso remoto no autorizado,
- enumeración de información sensible,
- ataques de fuerza bruta,
- escaladas de privilegios,
- y explotación remota de servicios vulnerables.

5.3.1 SSH inseguro

Se identificó que el servidor SSH permitía inicialmente:

- autenticación mediante contraseña,
- y acceso remoto utilizando cuentas privilegiadas.

Estas configuraciones representaban un riesgo crítico debido a la posibilidad de:

- ataques de fuerza bruta,
- reutilización de credenciales,
- y compromiso total del sistema mediante acceso root remoto.

La configuración insegura fue validada mediante revisión directa de parámetros SSH relacionados con:

- PermitRootLogin
- y PasswordAuthentication.

Posteriormente se aplicaron medidas de hardening para restringir estos mecanismos inseguros de autenticación.

5.3.2 FTP inseguro

El servidor FTP se encontraba expuesto mediante:

- el puerto 21/tcp,
- utilizando el servicio vsftpd.

Durante el análisis se verificó la configuración FTP y se identificó riesgo asociado a:

- transferencia de información en texto plano,
- exposición de credenciales,
- y posibles accesos inseguros.

Asimismo, se revisaron configuraciones relacionadas con:

- acceso anonymous,
- permisos de escritura,
- y autenticación de usuarios locales.

Debido al riesgo asociado al protocolo FTP sin cifrado, se decidió posteriormente:

- deshabilitar completamente el servicio,
- y sustituir el acceso remoto por mecanismos más seguros como SSH/SFTP.

5.3.3 Password débil WordPress/MySQL

Durante el análisis del archivo:

- wp-config.php,
se identificó una contraseña débil utilizada para la conexión entre WordPress y MariaDB.

La contraseña detectada:

- utilizaba un patrón simple,
- fácilmente reutilizable,
- y susceptible de ataques de fuerza bruta o diccionario.

La exposición de estas credenciales podía permitir:

- acceso completo a la base de datos,
- robo de información,
- modificación de contenido web,
- o compromiso total del entorno WordPress.

Posteriormente se sustituyó la contraseña vulnerable por una credencial robusta generada específicamente para el entorno.

5.3.4 Permisos inseguros wp-config.php

El archivo:

- wp-config.php,
contenía información crítica relacionada con:
- credenciales WordPress,
- acceso a MariaDB,
- y configuración sensible del entorno web.

Durante el análisis se identificó que el archivo requería restricciones adicionales de acceso para evitar:

- lectura no autorizada,
- exposición de credenciales,
- y acceso indebido por parte de usuarios no privilegiados.

Como medida de mitigación se aplicaron permisos restrictivos:

- 0600,
permitiendo acceso únicamente al propietario del archivo.

5.3.5 XMLRPC habilitado

Se detectó que WordPress mantenía habilitado el componente:

- xmlrpc.php.

Este endpoint permite funcionalidades de autenticación remota y comunicación externa, pero también es frecuentemente utilizado para:

- ataques de fuerza bruta,
- enumeración de usuarios,
- abuso de pingbacks,
- y automatización de ataques sobre WordPress.

La exposición de XMLRPC incrementaba considerablemente el riesgo asociado a accesos remotos automatizados sobre el entorno web.

Como medida correctiva se procedió posteriormente a deshabilitar completamente el endpoint.

5.3.6 Apache Directory Listing

Durante el análisis del servidor Apache se identificó la posibilidad de listado automático de directorios mediante la configuración:

- Options Indexes.

Esta funcionalidad permitía potencialmente:

- enumerar archivos internos,
- identificar estructuras del servidor,
- y acceder a información sensible del entorno web.

La presencia de Directory Listing representaba un riesgo de divulgación de información y facilitaba tareas de reconocimiento para un atacante.

Posteriormente se aplicó hardening sobre Apache mediante:

- Options -Indexes +FollowSymLinks.

5.3.7 Servicios innecesarios expuestos

El análisis del sistema permitió identificar diversos servicios activos que no eran necesarios para la funcionalidad principal del servidor web.

Entre ellos:

- Avahi,
- CUPS,
- Exim4,
- y FTP.

La exposición de estos servicios incrementaba innecesariamente:

- la superficie de ataque,
- la cantidad de puertos accesibles,
- y las posibilidades de explotación remota.

Como parte del hardening se procedió a:

- deshabilitar servicios innecesarios,
- cerrar puertos asociados,
- y validar posteriormente la persistencia de las mitigaciones.

5.4 Riesgos asociados

Las vulnerabilidades identificadas durante el análisis representaban riesgos críticos para:

- la confidencialidad,
- integridad,
- y disponibilidad del sistema.

Entre los principales riesgos asociados destacaban:

- acceso remoto no autorizado,
- compromiso de credenciales,
- robo de información sensible,
- escalada de privilegios,
- exposición de servicios vulnerables,
- y automatización de ataques sobre WordPress.

Asimismo, la combinación de múltiples configuraciones inseguras aumentaba significativamente la probabilidad de compromiso total del entorno.

La superficie de ataque inicial del sistema era elevada debido a:

- servicios innecesarios,
- protocolos inseguros,
- y configuraciones por defecto vulnerables.

5.5 Proceso de explotación realizado

Durante el laboratorio se realizaron pruebas controladas de validación sobre las vulnerabilidades identificadas con el objetivo de:

- confirmar su existencia,
- evaluar el impacto potencial,
- y verificar la necesidad de aplicar mitigaciones.

Las pruebas incluyeron:

- enumeración de puertos,
- validación de servicios activos,
- comprobación de configuraciones SSH,
- revisión de credenciales WordPress/MariaDB,
- verificación de permisos inseguros,
- y análisis de componentes vulnerables de WordPress.

Asimismo, se verificó:

- la exposición de XMLRPC,
- la accesibilidad FTP,
- y el comportamiento del servidor Apache ante configuraciones inseguras.

Todas las pruebas fueron realizadas en un entorno controlado y documentadas mediante evidencias técnicas y capturas de pantalla.

5.6 Impacto potencial

Las vulnerabilidades detectadas podían permitir distintos escenarios de compromiso del sistema dependiendo del vector de ataque utilizado.

Entre los impactos potenciales identificados se encontraban:

- acceso no autorizado al servidor,
- robo de credenciales,
- compromiso de WordPress,
- acceso indebido a MariaDB,
- divulgación de información sensible,
- ejecución remota de acciones no autorizadas,
- y degradación de la seguridad general del entorno.

Además, la existencia simultánea de múltiples debilidades incrementaba significativamente la posibilidad de:

- compromiso total del servidor,
- persistencia de atacantes,
- y explotación automatizada mediante herramientas ofensivas.

5.7 Evidencias técnicas de explotación

Durante todo el proceso se generaron múltiples evidencias técnicas que permitieron documentar:

- vulnerabilidades identificadas,
- configuraciones inseguras,
- mitigaciones aplicadas,
- y validaciones posteriores.

Las evidencias recopiladas incluyeron:

- resultados de escaneos Nmap,
- configuraciones SSH,
- listados de puertos abiertos,
- comprobaciones de permisos,
- validaciones de XMLRPC,
- verificaciones de servicios deshabilitados,
- y capturas de pantalla del entorno WordPress.

Asimismo, se elaboraron documentos técnicos específicos relacionados con:

- mitigación,
- análisis forense,
- y validaciones post-hardening,
permitiendo mantener trazabilidad completa de todas las acciones realizadas durante el proyecto.

6. MITIGACIÓN Y HARDENING DEL SISTEMA

6.1 Hardening SSH

Como parte del proceso de hardening se reforzó la configuración del servicio SSH para reducir el riesgo de accesos remotos no autorizados.

Inicialmente el servidor permitía:

- autenticación mediante contraseña,
- y acceso remoto utilizando cuentas privilegiadas.

Estas configuraciones incrementaban considerablemente el riesgo de:

- ataques de fuerza bruta,
- reutilización de credenciales,
- y compromiso total del sistema.

Para mitigar estos riesgos se modificó la configuración SSH deshabilitando:

- el acceso remoto como root,
- y la autenticación basada en contraseñas.

Las medidas implementadas fueron:

- PermitRootLogin no
- PasswordAuthentication no

Posteriormente se validó técnicamente la correcta aplicación de la configuración endurecida mediante comprobaciones directas del servicio SSH.

6.2 Protección de credenciales y contraseñas

Durante el análisis de seguridad se identificó el uso de credenciales débiles en la configuración de WordPress y MariaDB.

La contraseña detectada presentaba un riesgo crítico debido a:

- simplicidad,
- reutilización potencial,
- y vulnerabilidad frente a ataques de fuerza bruta o diccionario.

Como medida correctiva:

- se sustituyó la contraseña vulnerable por una credencial robusta,
- aleatoria,
- y de mayor complejidad.

Asimismo, se reforzó la protección de archivos críticos que almacenaban credenciales sensibles del entorno web.

Estas acciones permitieron mejorar significativamente:

- la protección de autenticación,
- la confidencialidad de la información,
- y la resistencia frente a accesos no autorizados.

6.3 Restricción de permisos en wp-config.php

El archivo:

- wp-config.php,
contenía información crítica relacionada con:
- credenciales WordPress,
- acceso a MariaDB,
- y parámetros sensibles del entorno web.

Para evitar accesos indebidos y exposición de credenciales se aplicaron permisos restrictivos utilizando:

```
chmod 600 /var/www/html/wp-config.php
```

Tras aplicar la medida:

- únicamente el propietario del archivo podía leer y modificar el contenido,
- mientras que grupos y otros usuarios quedaron completamente restringidos.

La validación posterior confirmó permisos:

- 0600 (-rw-----).

Esta medida permitió proteger adecuadamente las credenciales críticas del entorno WordPress y aplicar el principio de mínimo privilegio.

6.4 Deshabilitación FTP y acceso anonymous

El servicio FTP representaba un riesgo de seguridad debido a:

- transmisión de credenciales en texto plano,
- exposición innecesaria del puerto 21/tcp,
- y posibles accesos inseguros.

Durante el análisis también se revisó la configuración relacionada con:

- acceso anonymous,
- permisos de escritura,
- y autenticación local.

Como medida de hardening:

- se deshabilitó completamente el servicio vsftpd,
- y se eliminaron mecanismos de acceso FTP inseguros.

Asimismo, se validó que:

- el puerto 21/tcp dejara de estar expuesto,
- y que el servicio no permaneciera activo tras reiniciar el sistema.

Posteriormente se recomendó sustituir FTP por mecanismos cifrados como:

- SSH/SFTP.

6.5 Deshabilitación XMLRPC

Durante el análisis del entorno WordPress se identificó que el componente:

- xmlrpc.php, permanecía habilitado y accesible desde el servidor web.

Este endpoint representa un riesgo frecuente en WordPress debido a su utilización en:

- ataques de fuerza bruta,
- enumeración de usuarios,
- abuso de pingbacks,
- y automatización de ataques remotos.

Como medida correctiva se procedió a:

- deshabilitar completamente el componente XMLRPC,
- renombrando el archivo expuesto para evitar su acceso público.

Posteriormente se verificó que:

- el endpoint dejara de estar accesible,
- y únicamente permanecieran copias deshabilitadas del archivo original.

Esta medida permitió reducir significativamente la superficie de ataque del entorno WordPress.

6.6 Hardening Apache (-Indexes)

Durante la revisión de configuración Apache se detectó la posibilidad de:

- listado automático de directorios,
- mediante la directiva:
 - Options Indexes.

Esta configuración podía facilitar:

- enumeración de archivos internos,
- exposición de información sensible,
- y reconocimiento del entorno web por parte de atacantes.

Como medida de hardening se modificó la configuración Apache aplicando:

Options -Indexes +FollowSymLinks

Tras aplicar esta configuración:

- el servidor dejó de mostrar automáticamente el contenido de directorios sin archivo índice,
- reduciendo la exposición de información interna del entorno web.

La validación posterior confirmó la correcta aplicación del hardening Apache.

6.7 Eliminación de servicios innecesarios

Durante el análisis del sistema se identificaron diversos servicios activos que no eran necesarios para el funcionamiento del servidor web.

Entre los principales servicios deshabilitados se encontraban:

- Avahi,
- CUPS,
- Exim4,
- y FTP.

La presencia de estos servicios incrementaba:

- la superficie de ataque,
- la cantidad de puertos expuestos,
- y el riesgo de explotación remota.

Como parte del hardening:

- se detuvieron los servicios,
- se deshabilitó su inicio automático,
- y se validó posteriormente que permanecieran inactivos tras reiniciar el sistema.

Estas acciones permitieron reducir considerablemente la exposición innecesaria del servidor.

6.8 Reducción de superficie de ataque

El conjunto de medidas de mitigación aplicadas permitió reducir significativamente la superficie de ataque inicial del entorno.

Entre las acciones realizadas destacaron:

- cierre de puertos innecesarios,
- deshabilitación de servicios inseguros,
- protección de archivos críticos,
- endurecimiento SSH,
- y eliminación de componentes vulnerables de WordPress.

Tras completar el proceso de hardening, el servidor quedó limitado únicamente a los servicios estrictamente necesarios:

- Apache,
- SSH endurecido,
- y MariaDB restringido a localhost.

La reducción de exposición permitió minimizar:

- vectores de ataque,
- posibilidades de explotación remota,
- y riesgos asociados al entorno web.

6.9 Persistencia de mitigaciones tras reinicio

Tras aplicar todas las medidas de seguridad se realizaron múltiples validaciones posteriores al reinicio del sistema para garantizar:

- persistencia de configuraciones,
- servicios deshabilitados,
- y mantenimiento del hardening implementado.

Durante las primeras verificaciones se detectó que algunos servicios:

- volvieron a aparecer activos tras reiniciar la máquina virtual, lo que obligó a repetir parte del proceso de mitigación y endurecimiento.

Posteriormente:

- se reaplicaron configuraciones,
- se validó el estado de los servicios,
- y se confirmó finalmente la persistencia correcta de todas las mitigaciones.

Las validaciones finales demostraron que:

- los servicios innecesarios permanecieron deshabilitados,
- los puertos vulnerables continuaron cerrados,
- y el entorno WordPress mantuvo su funcionamiento correcto tras reiniciar el sistema.

6.10 Validación técnica de mitigaciones

Una vez aplicadas todas las medidas de hardening se realizaron múltiples verificaciones técnicas para confirmar la correcta implementación de las mitigaciones.

Las validaciones realizadas incluyeron:

- comprobación de puertos abiertos,
- verificación de servicios deshabilitados,
- revisión de configuración SSH,
- validación de permisos seguros,
- comprobación de XMLRPC deshabilitado,
- y pruebas de disponibilidad del entorno WordPress.

Asimismo, se verificó:

- la persistencia de los cambios tras reiniciar la máquina virtual,
- y el correcto funcionamiento de Apache y WordPress tras completar el hardening.

Las evidencias técnicas y capturas obtenidas confirmaron que:

- la superficie de ataque fue reducida correctamente,
- los servicios vulnerables quedaron mitigados,
- y el sistema mantuvo disponibilidad operativa tras aplicar todas las medidas de seguridad.

7. VALIDACIONES Y PRUEBAS FINALES

7.1 Verificación de puertos cerrados

Tras aplicar las medidas de hardening se realizaron comprobaciones finales sobre los puertos expuestos del sistema con el objetivo de validar la reducción de superficie de ataque.

Inicialmente el servidor mantenía múltiples servicios activos y puertos innecesarios expuestos, incluyendo:

- FTP (21/tcp),
- Avahi (5353/udp),
- y CUPS (631/tcp).

Posteriormente se realizaron validaciones mediante comandos de inspección de sockets y servicios activos para confirmar el cierre de dichos puertos.

Las verificaciones finales confirmaron que:

- los puertos asociados a servicios innecesarios dejaron de estar accesibles,
- y únicamente permanecieron activos los servicios estrictamente necesarios para el funcionamiento del entorno.

El estado final del sistema mostró únicamente:

- Apache HTTP,
- SSH endurecido,
- y MariaDB restringido a localhost.

7.2 Verificación SSH endurecido

Tras modificar la configuración del servicio SSH se realizaron validaciones específicas para confirmar el correcto endurecimiento del acceso remoto.

Las comprobaciones permitieron verificar que:

- el acceso remoto mediante root quedó deshabilitado,
- y la autenticación basada en contraseñas fue eliminada correctamente.

La configuración validada confirmó:

- PermitRootLogin no
- PasswordAuthentication no

Estas verificaciones garantizaron que el servidor SSH redujera significativamente:

- el riesgo de ataques de fuerza bruta,
- reutilización de credenciales,
- y accesos privilegiados no autorizados.

7.3 Validación permisos seguros

Se realizaron verificaciones posteriores sobre el archivo:

- wp-config.php,
para confirmar la correcta aplicación de permisos restrictivos.

Las comprobaciones validaron que el archivo quedó configurado con permisos:

- 0600 (-rw-----).

Esto confirmó que:

- únicamente el propietario del archivo podía acceder a su contenido,
- mientras que grupos y otros usuarios quedaron completamente restringidos.

La validación permitió asegurar la protección adecuada de:

- credenciales WordPress,
- parámetros sensibles,
- y acceso a MariaDB.

Asimismo, se confirmó la persistencia de los permisos tras reiniciar el sistema.

7.4 Validación XMLRPC deshabilitado

Tras aplicar la mitigación sobre:

- xmlrpc.php,
se realizaron verificaciones posteriores para confirmar que el endpoint vulnerable dejó de estar accesible.

Durante las primeras comprobaciones se detectó que el archivo:

- volvió a aparecer activo, lo que obligó a repetir el proceso de mitigación y validación.

Posteriormente se aplicó nuevamente la deshabilitación del componente XMLRPC y se verificó que:

- únicamente permanecieran archivos deshabilitados,
- sin exposición pública del endpoint original.

La validación final confirmó:

- la eliminación efectiva del componente vulnerable,
- y la reducción de riesgos asociados a:
 - fuerza bruta,
 - pingbacks,
 - y autenticación remota automatizada.

7.5 Validación WordPress operativo

Tras completar todas las medidas de hardening se realizaron pruebas funcionales para confirmar que:

- Apache,
- PHP,
- y WordPress, continuaban operativos.

Inicialmente el servidor mostraba la página por defecto de Apache debido a la presencia del archivo:

- index.html, el cual tenía prioridad sobre:
- index.php.

Posteriormente se corrigió esta situación eliminando el archivo por defecto y reiniciando el servicio Apache.

Las validaciones finales confirmaron:

- correcta carga del entorno WordPress,
- funcionamiento operativo del servidor web,
- y disponibilidad del servicio tras aplicar todas las mitigaciones.

Asimismo, se verificó el acceso desde la máquina host utilizando la dirección IP del servidor Debian.

8. PLAN DE RESPUESTA A INCIDENTES (NIST SP 800-61)

8.1 Preparación

La fase de preparación tuvo como objetivo establecer medidas preventivas y procedimientos orientados a reducir el impacto de posibles incidentes de seguridad sobre la infraestructura analizada.

Como parte de esta etapa se realizaron:

- análisis iniciales de la superficie de ataque,

- identificación de activos críticos,
- revisión de servicios expuestos,
- y recopilación de evidencias técnicas del entorno.

Asimismo, se implementaron controles orientados a:

- endurecimiento del sistema,
- protección de credenciales,
- restricción de accesos,
- y reducción de servicios innecesarios.

Entre las principales medidas preventivas aplicadas destacaron:

- hardening SSH,
- protección de wp-config.php,
- deshabilitación de FTP,
- eliminación de XMLRPC,
- y reducción de puertos expuestos.

También se documentaron:

- configuraciones críticas,
- procedimientos de validación,
- y evidencias técnicas,
 permitiendo disponer de trazabilidad y soporte documental ante posibles incidentes futuros.

8.2 Identificación

La fase de identificación se centró en detectar:

- vulnerabilidades,
- configuraciones inseguras,
- servicios expuestos,
- y posibles indicadores de compromiso dentro del entorno Debian y WordPress.

Para ello se realizaron:

- escaneos de puertos,
- análisis de servicios,
- revisión de configuraciones,
- y validación de componentes vulnerables.

Durante el análisis se identificaron múltiples riesgos de seguridad, incluyendo:

- SSH inseguro,
- FTP expuesto,
- password débil WordPress/MySQL,
- permisos inseguros,
- XMLRPC habilitado,
- y servicios innecesarios activos.

Asimismo, se verificó:

- exposición innecesaria de puertos,
- autenticación remota insegura,
- y configuraciones por defecto vulnerables.

La identificación temprana de estas debilidades permitió priorizar posteriormente las medidas de mitigación y hardening aplicadas.

8.3 Contención

Una vez identificadas las vulnerabilidades, se aplicaron medidas inmediatas orientadas a contener los riesgos detectados y limitar la exposición del sistema.

Las principales acciones de contención implementadas fueron:

- deshabilitación de servicios innecesarios,
- cierre de puertos vulnerables,
- restricción de accesos remotos,
- y endurecimiento de configuraciones críticas.

Entre las medidas aplicadas destacaron:

- deshabilitación de FTP,
- bloqueo de autenticación SSH insegura,
- eliminación de acceso root remoto,
- y desactivación de XMLRPC en WordPress.

Asimismo, se limitaron:

- permisos de archivos sensibles,
- exposición de directorios web,
- y servicios auxiliares innecesarios.

Estas acciones permitieron reducir rápidamente la superficie de ataque y minimizar el riesgo de explotación remota.

8.4 Erradicación

Tras contener las vulnerabilidades identificadas, se procedió a eliminar las configuraciones inseguras y componentes que representaban riesgos para la infraestructura.

Durante esta fase se realizaron acciones orientadas a:

- corregir credenciales débiles,
- eliminar servicios vulnerables,
- reforzar permisos,
- y endurecer configuraciones del sistema y aplicaciones.

Las principales medidas de erradicación incluyeron:

- sustitución de contraseñas débiles,
- protección de wp-config.php,
- deshabilitación completa de XMLRPC,
- eliminación de servicios innecesarios,
- y hardening Apache mediante Options -Indexes.

Asimismo, se validó que:

- los servicios vulnerables permanecieran deshabilitados,
- los puertos dejaran de estar expuestos,
- y las configuraciones persistieran correctamente tras reiniciar el sistema.

La fase de erradicación permitió eliminar la mayoría de los vectores de ataque identificados durante el análisis inicial.

8.5 Recuperación

Una vez aplicadas las medidas de mitigación y erradicación se realizaron múltiples validaciones para garantizar:

- estabilidad del entorno,
- disponibilidad de servicios críticos,
- y funcionamiento correcto de WordPress y Apache.

Durante esta fase se comprobó:

- el correcto funcionamiento del servidor web,
- la disponibilidad del entorno WordPress,
- y la persistencia de las configuraciones de seguridad implementadas.

Asimismo, se realizaron:

- reinicios controlados del sistema,
- validaciones posteriores de puertos y servicios,
- y pruebas funcionales de acceso web.

Las verificaciones finales confirmaron que:

- WordPress permaneció operativo,
- Apache continuó funcionando correctamente,
- y las mitigaciones persistieron tras reiniciar la máquina virtual.

Esto permitió garantizar la recuperación segura y estable del entorno tras completar el proceso de hardening.

8.6 Lecciones aprendidas

El desarrollo del proyecto permitió identificar múltiples aspectos críticos relacionados con:

- administración segura de sistemas Linux,
- protección de servicios web,
- y gestión de vulnerabilidades.

Entre las principales lecciones aprendidas destacaron:

- la importancia de reducir la superficie de ataque,
- la necesidad de deshabilitar servicios innecesarios,
- y la relevancia de proteger adecuadamente credenciales y accesos remotos.

Asimismo, se comprobó que:

- configuraciones por defecto inseguras pueden representar riesgos críticos,
- pequeños errores de permisos pueden comprometer información sensible,
- y determinadas mitigaciones requieren validaciones posteriores para garantizar persistencia real.

También se evidenció la importancia de:

- realizar comprobaciones post-reinicio,

- mantener documentación técnica completa,
- y validar continuamente el estado de seguridad del entorno.

Finalmente, el proyecto permitió aplicar de forma práctica conceptos relacionados con:

- hardening,
- análisis forense,
- gestión de incidentes,
- y controles de seguridad alineados con buenas prácticas de ciberseguridad e ISO 27001.

9. IMPLEMENTACIÓN DEL SGSI BASADO EN ISO 27001

9.1 Objetivo del SGSI

El objetivo principal del Sistema de Gestión de Seguridad de la Información (SGSI) implementado en este proyecto fue establecer un conjunto de controles y medidas orientadas a proteger:

- la confidencialidad,
- integridad,
- y disponibilidad de la información y los servicios del entorno analizado.

La implementación del SGSI se basó en principios inspirados en la norma ISO 27001, aplicando controles de seguridad sobre:

- sistemas Linux,
- servicios web,
- accesos remotos,
- credenciales,
- y protección de activos críticos.

Asimismo, el SGSI tuvo como finalidad:

- reducir la superficie de ataque,
- minimizar riesgos asociados a configuraciones inseguras,
- mejorar la postura de seguridad del entorno,
- y establecer procedimientos básicos de gestión de incidentes y hardening.

9.2 Alcance del SGSI

El alcance del SGSI incluyó todos los componentes críticos desplegados dentro del entorno Debian utilizado durante el proyecto.

Los sistemas y servicios incluidos fueron:

- servidor Debian Linux,
- Apache2,
- MariaDB,
- WordPress,
- servicio SSH,
- configuraciones de red,
- y servicios auxiliares instalados en la máquina virtual.

También se incluyeron dentro del alcance:

- archivos críticos,
- configuraciones sensibles,
- credenciales,
- y servicios expuestos mediante red.

El SGSI contempló:

- análisis de riesgos,
- identificación de vulnerabilidades,
- implementación de controles,
- validación de mitigaciones,
- y procedimientos básicos de respuesta ante incidentes.

9.3 Identificación de activos críticos

Durante el análisis se identificaron múltiples activos críticos cuya protección resultaba fundamental para garantizar la seguridad y operatividad del entorno.

Los principales activos identificados fueron:

Activo	Criticidad
Servidor Debian	Alta
Apache2	Alta
WordPress	Alta
MariaDB	Alta
Servicio SSH	Alta

Archivo wp-config.php	Crítica
Credenciales	Crítica
WordPress/MySQL	
Configuración Apache	Alta
Configuración SSH	Alta

Estos activos fueron considerados prioritarios debido a que un compromiso sobre ellos podía afectar directamente:

- la disponibilidad del servicio,
- la integridad de la información,
- y el acceso seguro al entorno.

9.4 Análisis de riesgos

Se realizó un análisis de riesgos orientado a identificar amenazas y vulnerabilidades asociadas a los activos críticos del entorno.

Durante el análisis se identificaron riesgos relacionados con:

- accesos remotos inseguros,
- exposición de servicios vulnerables,
- credenciales débiles,
- y configuraciones deficientes.

Entre los riesgos más relevantes detectados destacaron:

- ataques de fuerza bruta sobre SSH,
- acceso no autorizado a WordPress/MariaDB,
- explotación de XMLRPC,
- exposición de información mediante Apache Directory Listing,
- y explotación de servicios innecesarios activos.

Asimismo, la existencia de múltiples servicios expuestos incrementaba:

- la probabilidad de compromiso,
- la superficie de ataque,
- y el impacto potencial sobre la infraestructura.

El análisis permitió priorizar posteriormente las medidas de mitigación y hardening implementadas durante el proyecto.

9.5 Controles de seguridad implementados

Como parte de la implementación del SGSI se aplicaron múltiples controles de seguridad orientados a mitigar los riesgos identificados.

Entre los principales controles implementados destacaron:

- hardening SSH,
- deshabilitación de FTP,
- protección de credenciales,
- protección de archivos críticos,
- reducción de servicios innecesarios,
- y endurecimiento Apache y WordPress.

Las medidas aplicadas incluyeron:

- PermitRootLogin no
- PasswordAuthentication no
- chmod 600 wp-config.php
- deshabilitación XMLRPC
- Options -Indexes
- cierre de puertos innecesarios
- eliminación de servicios auxiliares vulnerables

Asimismo, se realizaron:

- validaciones posteriores,
- comprobaciones tras reinicio,
- y revisiones técnicas orientadas a garantizar la persistencia de las mitigaciones implementadas.

9.6 Políticas de seguridad propuestas

Como parte del SGSI se propusieron diversas políticas de seguridad orientadas a mantener la protección del entorno a largo plazo.

Las principales políticas recomendadas fueron:

- Política de mínimo privilegio.
- Política de gestión segura de contraseñas.
- Política de actualización y parcheo periódico.

- Política de hardening obligatorio de servicios.
- Política de control de accesos remotos.
- Política de monitorización y revisión de logs.
- Política de gestión de incidentes de seguridad.
- Política de backups y recuperación.
- Política de revisión periódica de vulnerabilidades.

Estas políticas buscan establecer una base organizativa orientada a:

- prevenir incidentes,
- reducir riesgos,
- y mantener controles de seguridad continuos sobre la infraestructura.

9.7 Gestión de accesos y mínimo privilegio

Durante el proyecto se aplicaron medidas orientadas a restringir accesos y limitar privilegios innecesarios sobre el entorno.

Las principales acciones implementadas fueron:

- deshabilitación de acceso root remoto,
- eliminación de autenticación SSH por contraseña,
- restricción de permisos sobre archivos críticos,
- y reducción de servicios accesibles desde red.

Asimismo, se aplicó el principio de mínimo privilegio mediante:

- permisos restrictivos en wp-config.php,
- limitación de accesos a MariaDB desde localhost,
- y reducción de usuarios con acceso privilegiado.

Estas medidas permitieron minimizar:

- el riesgo de escalada de privilegios,
- accesos indebidos,
- y compromiso de información sensible.

9.8 Protección de la información

La protección de la información se centró principalmente en:

- credenciales,
- archivos sensibles,
- configuraciones críticas,
- y servicios expuestos.

Como parte de esta protección se aplicaron:

- permisos restrictivos,
- endurecimiento SSH,
- reducción de servicios,
- y controles sobre WordPress y Apache.

Además, se recomendó:

- implementar copias de seguridad periódicas,
- cifrado de información sensible,
- y monitorización continua de accesos y eventos de seguridad.

Las medidas implementadas permitieron mejorar significativamente:

- la confidencialidad,
- integridad,
- y disponibilidad de la información del entorno.

9.9 Mejora continua del SGSI

Siguiendo los principios de mejora continua establecidos por ISO 27001, el proyecto contempló la necesidad de realizar:

- revisiones periódicas,
- validaciones continuas,
- y actualizaciones de seguridad sobre la infraestructura.

Durante el desarrollo del laboratorio se comprobó la importancia de:

- validar persistencia tras reinicios,
- revisar continuamente servicios activos,
- y repetir comprobaciones posteriores al hardening.

Como recomendaciones de mejora continua se propusieron:

- auditorías periódicas,
- escaneos regulares de vulnerabilidades,
- monitorización de logs,

- actualización constante de servicios,
- y revisión continua de políticas de seguridad.

El objetivo de estas acciones es mantener una postura de seguridad estable y adaptada frente a nuevas amenazas y vulnerabilidades futuras.

10. PLAN DE RECUPERACIÓN Y CONTINUIDAD

10.1 Estrategia de backups

Como parte de las medidas de continuidad y recuperación se estableció una estrategia básica de copias de seguridad orientada a proteger:

- configuraciones críticas,
- credenciales,

- archivos sensibles,
- y servicios esenciales del entorno.

Los principales elementos considerados para backup fueron:

- archivos de configuración de Apache,
- configuraciones SSH,
- archivos WordPress,
- base de datos MariaDB,
- y documentos de evidencias y validaciones generados durante el proyecto.

Asimismo, se realizaron copias de respaldo de archivos críticos como:

- wp-config.php,
antes de aplicar modificaciones de hardening y mitigación.

La estrategia propuesta contempla:

- realización periódica de backups,
- almacenamiento seguro de copias,
- y validación regular de integridad de las copias de seguridad.

El objetivo principal de esta estrategia es garantizar:

- recuperación rápida ante incidentes,
- continuidad operativa,
- y minimización de pérdida de información.

10.2 Recuperación ante incidentes

El proceso de recuperación ante incidentes se diseñó siguiendo un enfoque orientado a:

- restaurar la operatividad del entorno,
- contener el impacto del incidente,
- y garantizar la estabilidad del sistema tras aplicar medidas correctivas.

En caso de compromiso o fallo crítico, las acciones propuestas incluyen:

- aislamiento del sistema afectado,
- identificación del incidente,
- revisión de logs y evidencias,
- restauración de configuraciones seguras,
- y validación posterior de servicios críticos.

Asimismo, se contempla:

- restaurar backups verificados,
- reaplicar configuraciones de hardening,
- y comprobar nuevamente el estado de puertos y servicios.

Durante el proyecto se aplicaron procedimientos similares tras detectar:

- reactivación inesperada de servicios,
- y pérdida de persistencia en algunas mitigaciones tras reiniciar el sistema.

Estas validaciones permitieron reforzar el proceso de recuperación y garantizar la correcta aplicación final de las medidas de seguridad.

10.3 Restauración de servicios críticos

Como parte del plan de continuidad se definieron procedimientos orientados a restaurar los servicios esenciales del entorno tras un incidente o fallo del sistema.

Los servicios considerados críticos fueron:

- Apache2,
- WordPress,
- MariaDB,
- y SSH.

La restauración de servicios contempla:

- verificación de integridad de configuraciones,
- restauración de archivos críticos desde backup,
- validación de permisos,
- y comprobación de disponibilidad de servicios.

Durante las pruebas realizadas se verificó específicamente:

- el correcto funcionamiento de Apache,
- la disponibilidad del entorno WordPress,
- y la persistencia de configuraciones seguras tras reiniciar la máquina virtual.

Asimismo, se confirmó que:

- los servicios mitigados permanecieran deshabilitados,
- y únicamente se mantuvieran operativos los servicios estrictamente necesarios.

Estas acciones garantizan la continuidad operativa del entorno manteniendo simultáneamente un nivel adecuado de seguridad.

10.4 Validación post-recuperación

Tras completar las tareas de recuperación y restauración se realizaron múltiples validaciones técnicas orientadas a confirmar:

- estabilidad del sistema,
- persistencia de mitigaciones,
- y disponibilidad correcta de los servicios críticos.

Las validaciones realizadas incluyeron:

- comprobación de puertos abiertos,
- revisión de servicios activos,
- verificación de configuración SSH,
- validación de permisos seguros,
- comprobación de XMLRPC deshabilitado,
- y pruebas funcionales de WordPress.

Durante esta fase también se realizaron:

- reinicios controlados,
- verificaciones posteriores,
- y nuevas validaciones de persistencia.

Los resultados finales confirmaron que:

- las mitigaciones permanecieron activas,
- los servicios inseguros continuaron deshabilitados,
- y el entorno WordPress permaneció completamente operativo.

Asimismo, las evidencias obtenidas demostraron que:

- el sistema logró recuperarse correctamente,
- mantener disponibilidad operativa,
- y conservar las medidas de seguridad implementadas durante el hardening.

11. RECOMENDACIONES FUTURAS

11.1 Firewall y segmentación

Como medida adicional de seguridad se recomienda implementar un firewall que permita controlar y restringir el tráfico de red hacia los servicios expuestos del servidor.

La utilización de herramientas como:

- UFW,
 - iptables,
 - o firewalls perimetrales,
- permitiría limitar el acceso únicamente a:
- puertos necesarios,
 - direcciones IP autorizadas,
 - y servicios críticos del entorno.

Asimismo, se recomienda aplicar segmentación de red para separar:

- servicios web,
 - bases de datos,
 - y sistemas administrativos,
- reduciendo así el impacto potencial ante un compromiso de seguridad.

La segmentación también permitiría:

- limitar movimientos laterales,
- aislar servicios críticos,

- y mejorar el control del tráfico interno.

11.2 Autenticación mediante claves SSH

Aunque durante el proyecto se deshabilitó la autenticación SSH mediante contraseña, se recomienda complementar esta medida utilizando autenticación basada exclusivamente en claves públicas SSH.

El uso de claves SSH proporciona:

- mayor seguridad,
- resistencia frente a ataques de fuerza bruta,
- y mejor control sobre accesos remotos autorizados.

Asimismo, se recomienda:

- proteger las claves privadas mediante passphrase,
- restringir usuarios con acceso SSH,
- y monitorizar continuamente intentos de conexión remota.

Como medida adicional, también se podría implementar:

- Fail2Ban,
- autenticación multifactor (MFA),
- o restricciones por dirección IP.

Estas acciones reforzarían significativamente la seguridad del acceso remoto al servidor.

11.3 Monitorización y alertas

Se recomienda implementar mecanismos de monitorización continua orientados a detectar:

- actividad sospechosa,
- cambios no autorizados,
- y posibles incidentes de seguridad.

Entre las medidas recomendadas destacan:

- revisión periódica de logs,

- centralización de eventos,
- y generación automática de alertas de seguridad.

Asimismo, se podrían utilizar herramientas como:

- Fail2Ban,
 - Wazuh,
 - OSSEC,
 - o sistemas SIEM,
- para mejorar las capacidades de detección y respuesta ante incidentes.

La monitorización continua permitiría:

- identificar intentos de intrusión,
- detectar cambios anómalos,
- y responder de forma temprana ante amenazas.

11.4 Escaneos periódicos

Se recomienda realizar escaneos periódicos de seguridad y vulnerabilidades para identificar:

- nuevos servicios expuestos,
- configuraciones inseguras,
- software desactualizado,
- y posibles debilidades del entorno.

Estas revisiones deberían incluir:

- escaneos de puertos,
- análisis de servicios,
- revisión de configuraciones,
- y validación de permisos críticos.

Herramientas como:

- Nmap,
 - Nikto,
 - Lynis,
 - OpenVAS,
 - o Nessus,
- permitirían automatizar parte de estas tareas de auditoría y hardening.

La realización periódica de escaneos ayuda a:

- mantener la postura de seguridad,
- detectar desviaciones de configuración,
- y prevenir nuevas exposiciones de riesgo.

11.5 Gestión de actualizaciones

Se recomienda mantener actualizado todo el entorno mediante políticas periódicas de:

- actualización,
- parcheo,
- y revisión de versiones instaladas.

La gestión de actualizaciones debe aplicarse especialmente sobre:

- Debian,
- Apache2,
- MariaDB,
- WordPress,
- plugins,
- y servicios expuestos.

El uso de software desactualizado puede facilitar:

- explotación de vulnerabilidades conocidas,
- ejecución remota de código,
- y compromiso completo del sistema.

Asimismo, se recomienda:

- eliminar servicios y paquetes innecesarios,
- revisar dependencias instaladas,
- y validar actualizaciones en entornos controlados antes de aplicarlas en producción.

11.6 Auditorías de seguridad periódicas

Como medida de mejora continua se recomienda realizar auditorías periódicas de seguridad orientadas a evaluar:

- el estado del hardening,
- la exposición de servicios,
- la persistencia de configuraciones seguras,

- y el cumplimiento de políticas de seguridad.

Estas auditorías deberían incluir:

- revisiones técnicas,
- validaciones post-reinicio,
- análisis de permisos,
- pruebas de acceso remoto,
- y revisión de logs y eventos de seguridad.

Asimismo, se recomienda mantener:

- documentación actualizada,
- evidencias técnicas,
- y procedimientos de respuesta ante incidentes.

La realización continua de auditorías permite:

- detectar nuevas vulnerabilidades,
- validar controles implementados,
- y mantener una postura de seguridad alineada con buenas prácticas e ISO 27001.

12. CONCLUSIONES FINALES

El desarrollo del presente proyecto permitió aplicar de forma práctica conocimientos relacionados con:

- análisis forense,
- identificación de vulnerabilidades,
- hardening de sistemas Linux,
- protección de aplicaciones web,
- y gestión básica de incidentes de seguridad.

Durante las distintas fases del laboratorio se identificaron múltiples configuraciones inseguras y servicios vulnerables que comprometían la seguridad del entorno Debian y del ecosistema WordPress desplegado sobre Apache2 y MariaDB.

Entre las principales debilidades detectadas destacaron:

- autenticación SSH insegura,
- exposición de FTP,
- credenciales débiles,
- permisos inseguros sobre archivos críticos,
- XMLRPC habilitado,
- y servicios innecesarios activos.

Tras el análisis inicial se aplicaron múltiples medidas de mitigación orientadas a:

- reducir la superficie de ataque,
- proteger credenciales sensibles,
- restringir accesos,
- endurecer servicios críticos,
- y eliminar componentes vulnerables del sistema.

Asimismo, se realizaron:

- validaciones técnicas,
 - pruebas funcionales,
 - comprobaciones post-reinicio,
 - y verificaciones de persistencia,
- permitiendo confirmar que las mitigaciones implementadas permanecían activas y no afectaban la disponibilidad operativa del entorno.

Uno de los aspectos más relevantes del proyecto fue la validación continua de las medidas aplicadas, ya que durante el proceso se detectó que determinados servicios y configuraciones volvían a activarse tras reiniciar la máquina virtual. Esto permitió comprender la importancia de:

- validar persistencia,
- revisar configuraciones del sistema,
- y realizar auditorías continuas tras aplicar medidas de hardening.

El proyecto también permitió aplicar conceptos alineados con:

- NIST SP 800-61,
 - gestión de incidentes,
 - y principios básicos de ISO 27001,
- incluyendo:
- análisis de riesgos,
 - identificación de activos críticos,
 - controles de seguridad,
 - y mejora continua del SGSI.

Como resultado final, el entorno quedó significativamente más protegido, manteniendo únicamente los servicios estrictamente necesarios y reduciendo considerablemente los vectores de ataque inicialmente presentes.

Finalmente, el proyecto demostró la importancia de:

- aplicar hardening continuo,
- mantener políticas de seguridad adecuadas,
- proteger credenciales y accesos remotos,
- realizar auditorías periódicas,
- y validar constantemente el estado de seguridad de la infraestructura.

La combinación de:

- análisis técnico,
 - mitigación práctica,
 - validaciones reales,
 - documentación,
 - y evidencias obtenidas,
- permitió desarrollar una aproximación completa y profesional a un proceso real de análisis y fortalecimiento de seguridad sobre un entorno Linux y WordPress.

13. ANEXOS

13.1 Documentación técnica y evidencias

Como parte del proyecto se generó documentación técnica complementaria con el objetivo de mantener trazabilidad, soporte documental y evidencias reales del proceso de análisis, explotación, mitigación y validación del entorno.

Toda la información técnica asociada al proyecto se encuentra organizada dentro del repositorio Git del proyecto final, incluyendo:

- evidencias forenses,
- resultados de validación,
- mitigaciones aplicadas,
- comandos ejecutados,
- capturas de pantalla,
- scripts utilizados,
- y documentación técnica complementaria.

Los anexos incluidos permiten verificar técnicamente todas las acciones realizadas durante el laboratorio y proporcionan soporte documental del proceso completo de hardening y respuesta ante incidentes.

13.2 Evidencias forenses

Se incorporan documentos de evidencias generados durante la fase de análisis forense del entorno vulnerable, incluyendo:

- enumeración de puertos y servicios,
- revisión de configuraciones inseguras,
- logs del sistema,
- validaciones SSH,
- exposición de servicios vulnerables,
- y revisión de actividad relevante del sistema.

Documento asociado:

- Inicial_evidencia_forense.txt

13.3 Evidencias de mitigación y hardening

Se incluyen documentos técnicos que contienen:

- medidas de hardening aplicadas,
- comandos ejecutados,
- validaciones posteriores,
- configuración segura de servicios,
- endurecimiento SSH,
- deshabilitación de servicios innecesarios,
- protección de WordPress,
- mitigación de XMLRPC,
- y reducción de superficie de ataque.

Documentos asociados:

- mitigacion_aplicada_documentada.txt
- doble_verificacion_mitigaciones.txt

13.4 Capturas de pantalla

El repositorio Git del proyecto incluye múltiples capturas de pantalla utilizadas como soporte visual y evidencia técnica del proceso realizado.

Las capturas documentan:

- validaciones de puertos,
- configuraciones SSH,
- servicios deshabilitados,
- funcionamiento de WordPress,
- mitigaciones aplicadas,
- validaciones post-reinicio,
- y pruebas finales del entorno endurecido.

13.5 Scripts utilizados

Durante el proceso de mitigación y hardening se desarrollaron scripts orientados a automatizar tareas de:

- validación,
- endurecimiento,
- mitigación de vulnerabilidades,
- y generación de evidencias técnicas.

Debido a diferencias de formato entre sistemas Linux y Windows, algunos scripts fueron conservados directamente dentro del entorno Linux y referenciados en el repositorio del proyecto.

13.6 Repositorio del proyecto

Toda la documentación generada durante el proyecto fue organizada dentro de un repositorio Git creado específicamente para centralizar:

- evidencias,
- documentación técnica,
- capturas,
- scripts,
- validaciones,
- y resultados finales del laboratorio.

El repositorio actúa como soporte documental integral del proyecto de pentesting, hardening y análisis de seguridad realizado sobre el servidor Debian con WordPress.